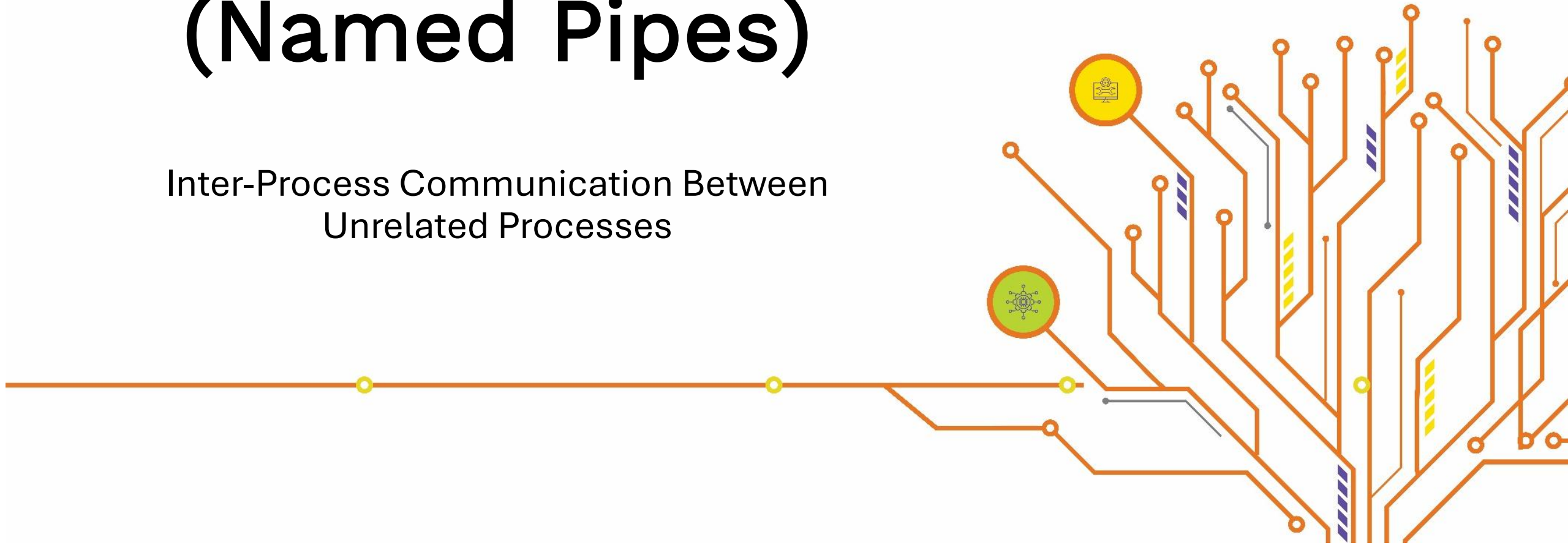
KPIT | APEX LAB

POSIX FIFOs (Named Pipes)

Inter-Process Communication Between Unrelated Processes



Disclaimer

This material has been developed exclusively for educational and academic purposes under the KPIT APEX Lab – Academic CSR Initiative. It is intended strictly for student learning, demonstrations, and classroom use.

Users are advised to verify and confirm the correctness of the contents before use.

It must not be copied, forwarded, shared, reproduced, or redistributed to any other individuals, groups, institutions, or platforms without prior written permission.

All examples, commands, diagrams, and demonstrations included here are meant solely to support conceptual understanding and learning.

Any use of these contents in commercial products, paid workshops, training programs, or proprietary documentation is strictly prohibited.

What is a FIFO?

- FIFO = First In, First Out
- A named pipe stored in the filesystem
- Enables communication between unrelated processes
- Works like a pipe, but has a path name (e.g., /tmp/myfifo)
- Unidirectional by default (one-way)

Characteristics of FIFOs

- Persistent object in filesystem until deleted
- Created using mkfifo()
- Accessed using open(), read(), write()
- Data flows sequentially (byte stream)
- Blocking behavior similar to pipes

Creating a FIFO (mkfifo)

- Prototype

```
int mkfifo(const char *pathname, mode_t mode);
```

- Example

```
mkfifo("/tmp/myfifo", 0666);
```

- Appears as a special file
- Remains until manually removed (unlink())

Opening a FIFO

- Writer

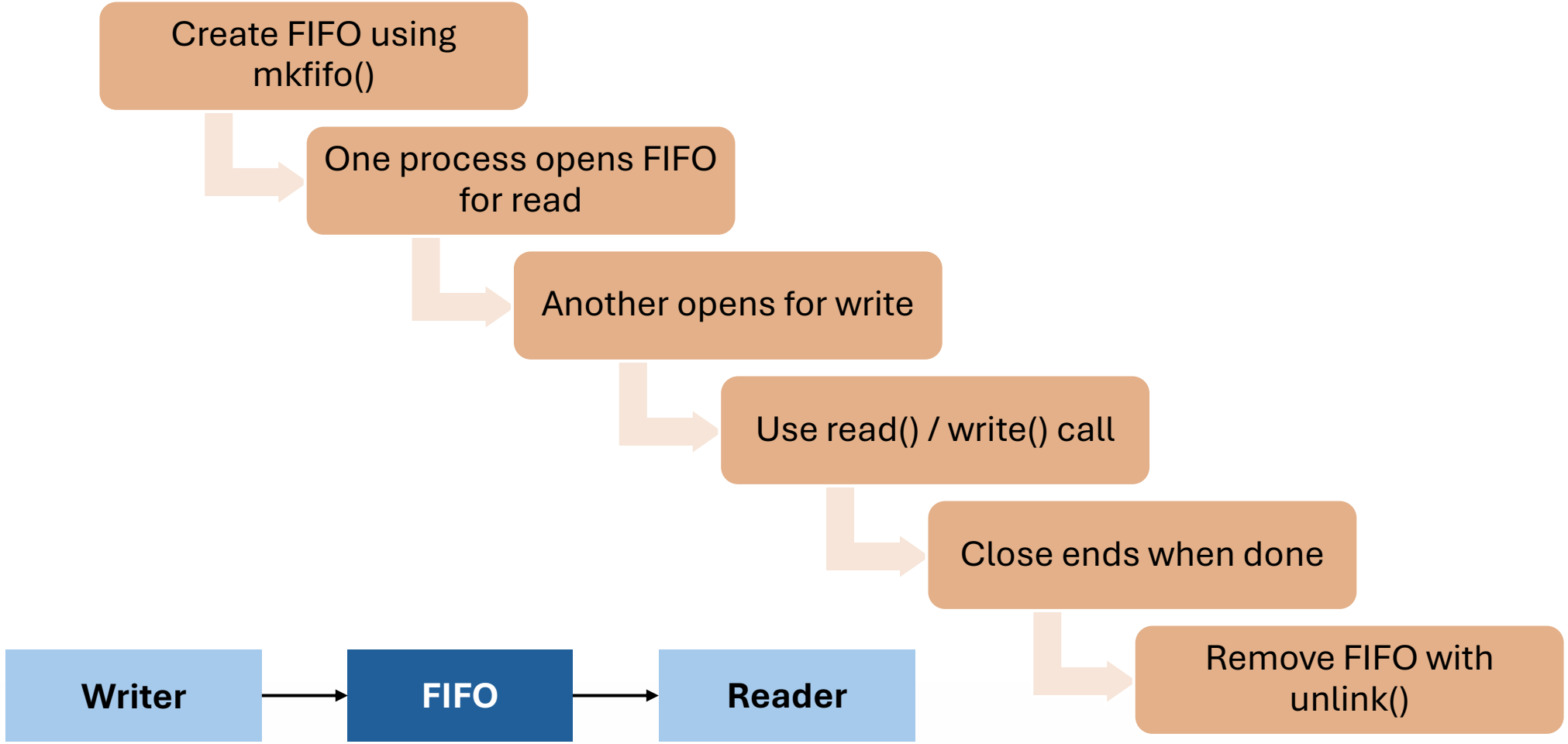
```
int fd = open("/tmp/myfifo", O_WRONLY);
```

- Reader

```
int fd = open("/tmp/myfifo", O_RDONLY);
```

- open() blocks until the other side opens
- Use O_NONBLOCK for non-blocking behavior

FIFO Communication Flow



Example: FIFO Writer & Reader

Writer blocks until a reader opens FIFO.

```
int fd = open("/tmp/myfifo", O_WRONLY);
write(fd, "Hello FIFO", 10);
close(fd);
```

```
int fd = open("/tmp/myfifo", O_RDONLY);
read(fd, buffer, sizeof(buffer));
close(fd);
```

Blocking Behavior

Writer blocks when:	Reader blocks when:	Reader gets 0 bytes (EOF) when:
• No reader has opened the FIFO	• No writer has opened the FIFO • FIFO empty (but writer still open)	• All writers have closed the FIFO

FIFO vs Anonymous Pipe

Feature	FIFO	Anonymous Pipe
Has name	Yes	No
Visible in filesystem	Yes	No
Works between	Any processes	Related processes only
Created by	mkfifo()	pipe()
Lifetime	Until deleted	Until all ends closed
Direction	One way	One way

Typical Use Cases

- Communication between apps in different terminals
- Producer/consumer programs
- Logging processes
- Simple command pipelines without shell
- Embedded systems: sensor reader ↔ logger tool

Example 1: FIFO Reader + Writer (1/4)

- **Instructions:**
  - Terminal 1 → run reader program
  - Terminal 2 → run writer program
  - Observe blocking and output
  - Delete FIFO using unlink()
- **Expected output:**

  Reader: Received → Hello FIFO

Example 1: FIFO Reader + Writer (2/4)

```
/* fifo_common.h – shared constants */

#ifndef FIFO_COMMON_H
#define FIFO_COMMON_H

#define FIFO_PATH "/tmp/my_fifo" /* <-- change if you
wish */
#define MSG "Hello FIFO" /* message the writer sends */

#endif /* FIFO_COMMON_H */
```

Example 1: (3/4)

```c
/* reader.c */
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include "fifo_common.h"

int main(void){
/* Create the FIFO if it does not exist yet */
if (mkfifo(FIFO_PATH, 0666) == -1) {
        perror("mkfifo");
        /* If the FIFO already exists we can ignore
        the error */
}
```

```c
/* Open FIFO for reading – this call blocks until a writer
opens it */
int fd = open(FIFO_PATH, O_RDONLY);
if (fd == -1) {perror("open");exit(EXIT_FAILURE);}

char buf[256] = {0};
ssize_t n = read(fd, buf, sizeof(buf)-1);
if (n == -1) {
        perror("read");
        close(fd);
        exit(EXIT_FAILURE);
}

printf("Reader: Received → %s\n", buf);

close(fd);
/* Remove the FIFO – optional; you can also do it from a
third program */
if (unlink(FIFO_PATH) == -1) {
                perror("unlink");
}

return 0;
}
```

Example 1: (4/4)

```c
/* writer.c */
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include "fifo_common.h"

int main(void)
{
/* Open FIFO for writing – this will block until a
reader has opened it */
int fd = open(FIFO_PATH, O_WRONLY);
if (fd == -1) {
        perror("open");
        exit(EXIT_FAILURE);
}
```

```c
ssize_t w = write(fd, MSG, sizeof(MSG)); /*
include terminating '\0' */
if (w == -1) {
        perror("write");
        close(fd);
        exit(EXIT_FAILURE);
}

printf("Writer: Sent → %s\n", MSG);
close(fd);
return 0;
}
```

Example 2: Multi-Message FIFO (1/6)

- **Task:**
- Writer sends multiple sensor values
- Reader prints each with timestamp
- Stop when writer closes FIFO
- **Learning:** EOF + streaming behavior

Example 2: Multi-Message FIFO (2/6)

```
/* fifo_common.h – shared constants */
#ifndef FIFO_COMMON_H
#define FIFO_COMMON_H

#define FIFO_PATH "/tmp/sensor_fifo" /* name of the FIFO */
#define NUM_VALUES 10 /* how many values the writer sends */
#define SLEEP_SEC 1 /* delay between two values (seconds) */

#endif /* FIFO_COMMON_H */
```

Example 2: Multi-Message FIFO (3/6)

```c
/* reader.c ----------------------------------------
----------------- */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <time.h>
#include <string.h>
#include "fifo_common.h"

#define LINE_MAX 128
 /* maximum length of a line from the FIFO */
```

```c
static void print_with_timestamp(const char *value)
{
	time_t now = time(NULL);
	struct tm tm_now;
	localtime_r(&now, &tm_now);
	char ts[32];
	strftime(ts, sizeof(ts), "%Y-%m-%d %H:%M:%S",
	&tm_now);
	printf("Reader [%s] → %s\n", ts, value);
	fflush(stdout);
}
```

Example 2: (4/6)

```
int main(void)
{
/* Open FIFO for reading – blocks until writer
opens it */
int fd = open(FIFO_PATH, O_RDONLY);
if (fd == -1) {
perror("open");
exit(EXIT_FAILURE);
}

/* Wrap the fd into a FILE* so we can use fgets()
(line-oriented I/O) */
FILE *fp = fdopen(fd, "r");
if (!fp) {
perror("fdopen");
close(fd);
exit(EXIT_FAILURE);
}
```

```
char line[LINE_MAX];
while (fgets(line, sizeof(line), fp) != NULL) {
        /* Strip trailing newline */
        line[strcspn(line, "\n")] = '\0';
        print_with_timestamp(line);
}

/* fgets() returned NULL – either error or EOF (writer
closed) */
if (feof(fp))
                printf("Reader: EOF reached – writer has closed
                                the FIFO.\n");
else
                perror("fgets");

fclose(fp); /* also closes the underlying fd */
/* Optional clean-up of the FIFO (remove the file) */
if (unlink(FIFO_PATH) == -1)
                perror("unlink");

return 0;
}
```

Example 2: Multi-Message FIFO (5/6)

```
/* writer.c ----------------------------------------
---------------- */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <time.h>
#include "fifo_common.h"

int main(void)
{
/* Create the FIFO if it does not exist yet */
if (mkfifo(FIFO_PATH, 0666) == -1) {
perror("mkfifo");
/* If the FIFO already exists we can ignore the
error */
}
```

```
/* Open the FIFO for writing – blocks until a reader
opens it */
int fd = open(FIFO_PATH, O_WRONLY);
if (fd == -1) {
perror("open");
exit(EXIT_FAILURE);
}
```

Example 2: Multi-Message FIFO (6/6)

```
/* Stream NUM_VALUES random sensor readings, one per line */
srand((unsigned)time(NULL));
for (int i = 0; i < NUM_VALUES; ++i) {
        int sensor = rand() % 1000; /* dummy sensor value */
        dprintf(fd, "%d\n", sensor); /* write value + newline */
        printf("Writer: sent value %d\n", sensor);
        fflush(stdout);
        sleep(SLEEP_SEC);
}

/* Closing the descriptor sends EOF to the reader */
close(fd);
printf("Writer: finished, FIFO closed → EOF will be seen by
reader.\n");
return 0;
}
```

Example 3: Chat Application using FIFO

- **Task:**
- Create two FIFOs: `fifo1`, `fifo2`
- Terminal 1 reads from fifo1, writes to fifo2
- Terminal 2 does the opposite
- Build a simple two-terminal chat

Common Pitfalls

- Forgetting to create FIFO before opening
- Opening write-only FIFO → blocks forever
- Not removing FIFO after use
- Using FIFO for large binary blobs (better use sockets/shared memory)
- Multiple writers → interleaving data

Troubleshooting

- Use ls -l to verify FIFO exists
- Use strace to debug blocking behavior
- Check open() flags
- Ensure permissions allow read/write
- Use unlink() to clean old FIFOs

Summary

- FIFOs are named pipes for process-to-process communication
- Work between unrelated processes
- Blocking/open rules must be understood
- Great for small IPC and teaching IPC concepts
- Simple, efficient alternative to sockets or message queues

Scenario: Multi-Terminal Command Dispatcher (1/3)

Context

- Create a command dispatcher that allows any terminal to send commands to a central daemon via a named pipe.
- The daemon executes simple actions and returns text responses over per-client response FIFOs.

Scenario: Multi-Terminal Command Dispatcher (2/3)

PART 1 — Daemon (Dispatcher)

- On startup: create main FIFO /tmp/cmd_fifo.
- Loop: open for read, receive commands of form:

  <client_fifo_name> <command> [args...]

- Supported commands: echo <text> → respond with <text>
- time → respond with current time
- rev <text> → respond with reversed text
- Open the provided <client_fifo_name> for write and send response.
- Robustness: ignore malformed messages; keep running.

Scenario: Multi-Terminal Command Dispatcher (3/3)

PART 2 — Client Tool

- Create a unique response FIFO, e.g., /tmp/resp_<pid>.
- Write request to /tmp/cmd_fifo with your response FIFO path.
- Open your response FIFO for read; print response; unlink it.